

← Go Back

Hardware

- Arduino UNO R4 WiFi CAN Bus
- Arduino UNO R4 WiFi User Manual
- Arduino UNO R4 WiFi Arduino Cloud Setup Guide
- Arduino UNO R4 WiFi Digital-to-Analog Converter (DAC)
- Debugging the Arduino UNO R4 WiFi
- Arduino UNO R4 WiFi EEPROM
- UNO R4 WiFi Custom Firmware Upload to ESP32 (Advanced)
- Using the Arduino UNO R4 WiFi LED Matrix
- Arduino UNO R4 WiFi OPAMP
- Arduino UNO R4 WiFi Qwiic Connector
- Getting Started with UNO R4 WiFi
- Arduino UNO R4 WiFi Real-Time Clock
- UNO R4 WiFi Stack Trace Debug Runtime Errors**
- UNO R4 Capacitive-Touch Tutorial
- Arduino UNO R4 WiFi USB HID
- Arduino UNO R4 WiFi VRTC & OFF Pins
- UNO R4 WiFi Network Examples

Home / Hardware / UNO R4 WiFi / **UNO R4 WiFi Stack Trace Debug Runtime Errors**

UNO R4 WiFi Stack Trace Debug Runtime Errors

Learn how to get information from the Cortex-M4 processor on a runtime error.

Author: Sebastian Wikström · Last revision: 10/07/2025

The Arduino UNO R4 WiFi uses **CmBacktrace** to print useful information from the **Arm Cortex-M4** processor as serial output when a runtime error occurs. The output includes an `addr2line` command that can be used to produce a **stack trace** (also **stack backtrace**, or **stack traceback**), which can be used to find the source of the error.

In this tutorial, we will show to enable this feature.

Hardware & Software Needed

- Arduino IDE
- Arduino UNO R4 WiFi

View the Runtime Error

To get the runtime error information in the Serial Monitor, follow the steps below:

ON THIS PAGE

Hardware & Software Needed

- View the Runtime Error
- Generate a Stack Trace +
- Additional Instructions +

- 1 Ensure that a serial connection must be initiated before the error occurs, by calling `Serial.begin()` in your sketch. You can include this code inside the beginning of the `setup()` function:

```
1 Serial.begin(115200);  
2 while (!Serial);
```

- 2 Connect your UNO R4 WiFi board to your computer.
- 3 Open the Serial Monitor. Click the



Serial Monitor
icon

Serial Monitor button in the top-right corner or select **Tools > Serial Monitor** in the menu bar.

- 4 If you made any changes to your sketch, click the



Upload icon

Upload button. Otherwise, you can simply press the **RESET** button on the board.

- 5 If an exception occurs on the board, information will be displayed in the



Serial output.

- 6 Following the "Registers information" table, the type of fault will be specified. You can read more about the types [here](#).
- 7 The last line includes a command for the `addr2line` utility. See [Running addr2line for more information](#).

Generate a Stack Trace

You can use the `addr2line` tool to generate a stack trace. See the instructions below:

Note: `addr2line` is not available as a native Windows application, but can be run with [Windows Subsystem for Linux \(WSL\)](#).

The `addr2line` utility is included in the **Arduino UNO R4 Boards** boards package. However, running it in this way requires modifying the command included in the output. For convenience, you may want to install `addr2line` on your system.

To install `addr2line` (optional), use the OS specific instructions below:

Windows application, but can be run with [Windows](#)

[Subsystem for Linux \(WSL\)](#).

The Ubuntu distribution of Linux is installed by default and should come with

`addr2line`.

macOS: `addr2line` can be installed with [Homebrew](#) by running

`brew install binutils` in Terminal.

Linux: `addr2line` may already be installed on your system. Otherwise, run `apt-get install binutils` in Terminal (Ubuntu, Debian), or see [command-not-found.com/addr2line](#) for other distributions.

1. Copy the `addr2line` Command

Windows (WSL):

- 1 Copy the command from the serial output.
- 2 Paste the command into a text editor.
- 3 Replace `C:` with `/mnt/c` and replace all backslashes (`\`) with forward slashes (`/`).

macOS:

Copy the command from the serial output (no modification required).

Linux:

Copy the command from the serial output (no modification required).

If you don't want to install `addr2line`, you can use the `addr2line` from the board package. See the instructions [here](#)

2. Running the `addr2line` Command

Note: The sketch needs to have been compiled on the same computer you are running `addr2line` on.

Windows (WSL):

- 1 Open [Windows Powershell](#).
- 2 Type `Ubuntu` into Windows Powershell and press Enter.
- 3 Paste the modified command into Windows Powershell by right-clicking on the window.
- 4 Press Enter to run the command.

macOS:

- 1 Open Terminal.
- 2 Press $\text{⌘} + V$ to paste the command.
- 3 Press Enter to run the command.

Linux:

- 1 Open Terminal.
- 2 Press $\text{Ctrl} + \text{⇧Shift} + V$ to paste the command.
- 3 Press Enter to run the command.

readable format.

3. Reading the `addr2line` Output

By default, the command outputs the following for each function call:

```
address: " 0x00004188", "
0x0000426e", ... (remove the
-a flag if you don't want to
include these)
```

```
function name: "
_ZN4UART5writeEh", " loop
", ...
```

```
line number "
/Users/sebastianwikstrom
/Documents/Arduino/Error
Inducer/ErrorInducer.ino
:67
, ...
```

Follow these steps:

- 1 Look for the topmost line in the output that's inside your sketch. The number following the path is the line number where the error occurred. For example,

```
/Users/username/Docum
ents/Arduino/ErrorInd
ucer/ErrorInducer.ino
:67
```

indicates that the error occurred on line 67. By reading further down the output, you can step backward through the function calls that were made.



Reading the
addr2line

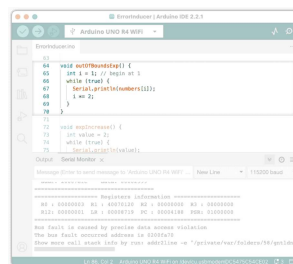
stack trace in
Terminal.

- 2 Open the sketch in Arduino IDE and find the line number from the previous step (the number is displayed to the left of each line).
- 3 Analyze the row where the error occurred and try to understand what may be triggering the error.

If you're not sure, use the `Serial.println()` function to output the values of any variables being used. Then upload the sketch again, and use the serial output to see what the states of those variables were before the error occurred.

To see from where the function was called, look at the preceding function call in the `addr2line` output.

In this example, an out-of-bounds access of the `numbers` array occurs after a few iterations of the `while(true)` loop:



Analyzing the code.

Additional Instructions

Copy the `addr2line` Command (Board Package)

 These instructions replace the instructions in [1. Copy `addr2line` Command](#)

If you want to use the `addr2line` directly from the board package, follow the instructions below

Windows (WSL):

- 1 Copy the command from the serial output.
- 2 Paste the command into a text editor.
- 3 Replace the word `addr2line` with

```
/mnt/c/Users/User/App  
data/Local/Arduino15/  
packages/arduino/tool  
s/arm-none-eabi-  
gcc/7-2017q4/bin/arm-  
none-eabi-addr2line
```
- 4 For the file path after the `-e` flag, replace `C:` with `/mnt/c` and replace all backslashes (`\`) with forward slashes (`/`).

macOS:

- 1 Copy the command from the serial output.
- 2 Paste the command into a text editor.
- 3 Replace the word `addr2line` with

```
~/Library/Arduino15/p  
ackages/arduino/tools  
/arm-none-eabi-gcc/7-  
2017q4/bin/arm-none-  
eabi-addr2line
```

Linux:

- 1 Copy the command from the serial output.
- 2 Paste the command into a text editor.
- 3 Replace the word `addr2line` with

```
.arduino15/packages/a  
rduino/tools/arm-  
none-eabi-gcc/7-  
2017q4/bin/arm-none-  
eabi-addr2line
```

Suggest changes	Need support?	License
The content on docs.arduino.cc is facilitated through a public GitHub repository. If you see anything wrong, you can edit this page here .	Help Center Ask the Arduino Forum Discover Arduino Discord	The Arduino documentation is licensed under the Creative Commons Attribution-Share Alike 4.0 license.

Was this article helpful?

